

01_preprocessing

June 24, 2026

1 Step 1: Preprocessing and Feature Construction for Authoritative v4

This notebook is the narrative companion to the frozen Step 1 artifacts in `data/processed/v4/`. Its job is to make the authoritative preprocessing layer legible: which feature families survive into the v4 matrix, how the **Control A** / **Control B** split is fixed, and which hashes and matrices are reused unchanged by the later benchmarking, transfer, SHAP, robustness, and sensitivity steps.

```
[1]: import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import polars as pl

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
            exists():
                return base
    raise RuntimeError('Could not locate repository root from the current_
working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))

STEP_TAG = 'step1'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
```

```

TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 10,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
COND_ORDER = ['pd', 'hd', 'als']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS', 'control': 'Control'}
COND_COLOR = {'pd': '#1b4f72', 'hd': '#196f3d', 'als': '#af601a', 'control': '
↪ #6c757d'}

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

def feature_family(feature_name: str) -> str:
    if feature_name in {'cv_stride', 'cv_swing'}:
        return 'variability'
    if feature_name == 'dfa_alpha_stride':
        return 'fractal'
    if 'asymmetry' in feature_name:

```

```

        return 'asymmetry'
    if feature_name.endswith('_pct'):
        return 'phase_percentage'
    return 'raw_timing'

from src.features import ALL_FEATURE_COLS

PROC_DIR = REPO_ROOT / 'data' / 'processed' / 'v4'
PROTOCOL = json.loads((PROC_DIR / 'v4_protocol_manifest.json').read_text())
PREPROC = json.loads((PROC_DIR / 'preprocessing_manifest_v4.json').read_text())
PARTITION = json.loads((PROC_DIR / 'control_partition_v4.json').read_text())
FEATURES = pl.read_csv(PROC_DIR / 'gait_features_v4.csv')
SUBJECT_LEVEL = pl.read_csv(PROC_DIR / 'gait_features_v4_subject_level.csv')
PER_STRIDE = pl.read_csv(PROC_DIR / 'gait_features_v4_per_stride_only.csv')
DFA_SENS = pl.read_csv(PROC_DIR / 'gait_features_v4_dfa_sensitivity.csv')
NO_DFA_SENS = pl.read_csv(PROC_DIR / 'gait_features_v4_no_dfa_sensitivity.csv')
TIMING_SENS = pl.read_csv(PROC_DIR / 'gait_features_v4_timing_sensitivity.csv')
print({'main_shape': FEATURES.shape, 'subject_level_shape': SUBJECT_LEVEL.
    ↪shape, 'n_features': PREPROC['n_features']})

```

```
{'main_shape': (13765, 19), 'subject_level_shape': (63, 18), 'n_features': 14}
```

1.1 Feature-family inventory, manifest hashes, and frozen control membership

The Step 1 tables are the provenance backbone for the downstream notebooks. The retained representation contains **14 features** in total, dominated by **raw timing** (7 features), followed by **phase-percentage** (3), **variability** (2), and one feature each in **asymmetry** and **fractal** structure. The control-partition table makes the frozen **8-vs-8** Control A / Control B split explicit, while the hash summary records the immutable preprocessing surfaces that every authoritative downstream result should match.

```

[2]: feature_family_df = pd.DataFrame({
    'feature': ALL_FEATURE_COLS,
    'family': [feature_family(feature) for feature in ALL_FEATURE_COLS],
})
family_counts = feature_family_df.groupby('family', as_index=False).size().
    ↪rename(columns={'size': 'n_features'})
save_table_csv(family_counts, 'step1_feature_family_table.csv')

hash_summary = pd.DataFrame([
    {'artifact': 'protocol_manifest', 'sha256': ↪
    ↪PREPROC['protocol_manifest_sha256']},
    {'artifact': 'feature_matrix', 'sha256': PREPROC['feature_matrix_sha256']},
    {'artifact': 'partition', 'sha256': PREPROC['partition_sha256']},
    {'artifact': 'subject_level_matrix', 'sha256': ↪
    ↪PREPROC['subject_level_matrix_sha256']},

```

```

    {'artifact': 'per_stride_only_matrix', 'sha256':␣
↪PREPROC['per_stride_only_matrix_sha256']},
    {'artifact': 'timing_sensitivity_matrix', 'sha256':␣
↪PREPROC['timing_sensitivity_matrix_sha256']},
    {'artifact': 'dfa_sensitivity_matrix', 'sha256':␣
↪PREPROC['dfa_sensitivity_matrix_sha256']},
    {'artifact': 'no_dfa_sensitivity_matrix', 'sha256':␣
↪PREPROC['no_dfa_sensitivity_matrix_sha256']},
])
save_table_csv(hash_summary, 'step1_hash_summary.csv')
partition_summary = pd.concat(
    [pd.DataFrame({'subject_id': PARTITION['control_A'], 'group':␣
↪'control_A'}), pd.DataFrame({'subject_id': PARTITION['control_B'], 'group':␣
↪'control_B'})],
    ignore_index=True,
)
save_table_csv(partition_summary, 'step1_control_partition_summary.csv')
feature_family_df

```

Saved table: report/tables/v4/step1_feature_family_table.csv

Saved table: report/tables/v4/step1_hash_summary.csv

Saved table: report/tables/v4/step1_control_partition_summary.csv

```

[2]:
      feature      family
0    left_stride_s  raw_timing
1    right_stride_s  raw_timing
2    left_swing_s   raw_timing
3    right_swing_s  raw_timing
4    left_swing_pct phase_percentage
5    right_swing_pct phase_percentage
6    left_stance_s  raw_timing
7    right_stance_s raw_timing
8    double_support_s raw_timing
9    double_support_pct phase_percentage
10   asymmetry_index  asymmetry
11      cv_stride     variability
12      cv_swing     variability
13   dfa_alpha_stride    fractal

```

1.2 Feature-family and Control A / Control B figures

These figures support different parts of the methods story. **step1_feature_family_inventory** shows that the v4 representation is still timing-heavy, which becomes relevant later when Step 4 SHAP diagnostics and Step 5 sensitivity analyses repeatedly point back to timing and variability families. **step1_control_split_summary** is the visual anchor for the disjoint-control design: it explains why cross-condition transfer can be evaluated against a held-out control partition and why Step 8 can revisit the same question under alternate near-optimal splits.

```

[3]: fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 3.3),
    ↪constrained_layout=True)
family_palette = {
    'asymmetry': '#7D3C98',
    'fractal': '#AF601A',
    'phase_percentage': '#117A65',
    'raw_timing': '#1F618D',
    'variability': '#566573',
}
colors = [family_palette.get(family, '#5D6D7E') for family in
    ↪family_counts['family']]
bars = ax.bar(
    family_counts['family'],
    family_counts['n_features'],
    color=colors,
    edgecolor='black',
    linewidth=0.5,
)
ax.set_title('Feature-family inventory')
ax.set_ylabel('Feature count')
ax.tick_params(axis='x', rotation=20)
ax.grid(axis='y', alpha=0.2)
ymax = float(family_counts['n_features'].max())
ax.set_ylim(0, ymax + 1.55)
for idx, (_, row) in enumerate(family_counts.iterrows()):
    ax.text(idx, row['n_features'] + 0.16, str(int(row['n_features'])),
    ↪ha='center', va='bottom', fontsize=8)
savefig_stem(fig, 'step1_feature_family_inventory')
plt.show()

control_a = PREPROC['control_partition_summary']['control_A']
control_b = PREPROC['control_partition_summary']['control_B']
fig, axes = plt.subplots(1, 2, figsize=(IEEE_TWO_COL * 0.78, 3.3),
    ↪constrained_layout=True)
bar_colors = ['#D6EAF8', '#AED6F1']
for ax, metric_key, title in [
    (axes[0], 'mean_age_years', 'Mean age (years)'),
    (axes[1], 'mean_gait_speed_m_per_s', 'Mean gait speed (m/s)'),
]:
    values = [control_a[metric_key], control_b[metric_key]]
    bars = ax.bar(['Control A', 'Control B'], values, color=bar_colors,
    ↪edgecolor='black', linewidth=0.5)
    ax.set_title(title)
    ax.grid(axis='y', alpha=0.2)
    ax.set_ylim(0, max(values) * 1.22)
    for bar, value in zip(bars, values):

```

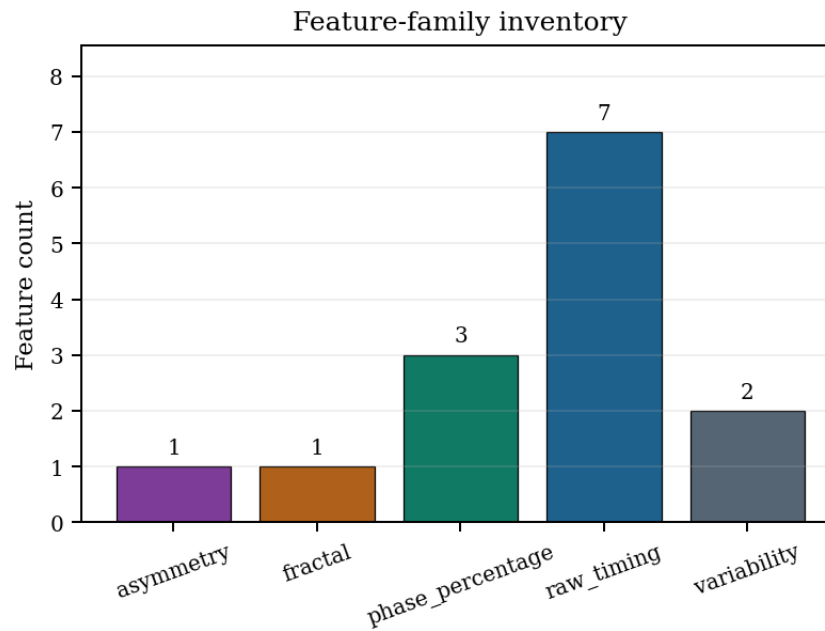
```

        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() +
↪max(values) * 0.03, f'{value:.2f}', ha='center', va='bottom', fontsize=8)
fig.suptitle('Balanced Control A / Control B split summary', y=1.04)
savefig_stem(fig, 'step1_control_split_summary')
plt.show()

```

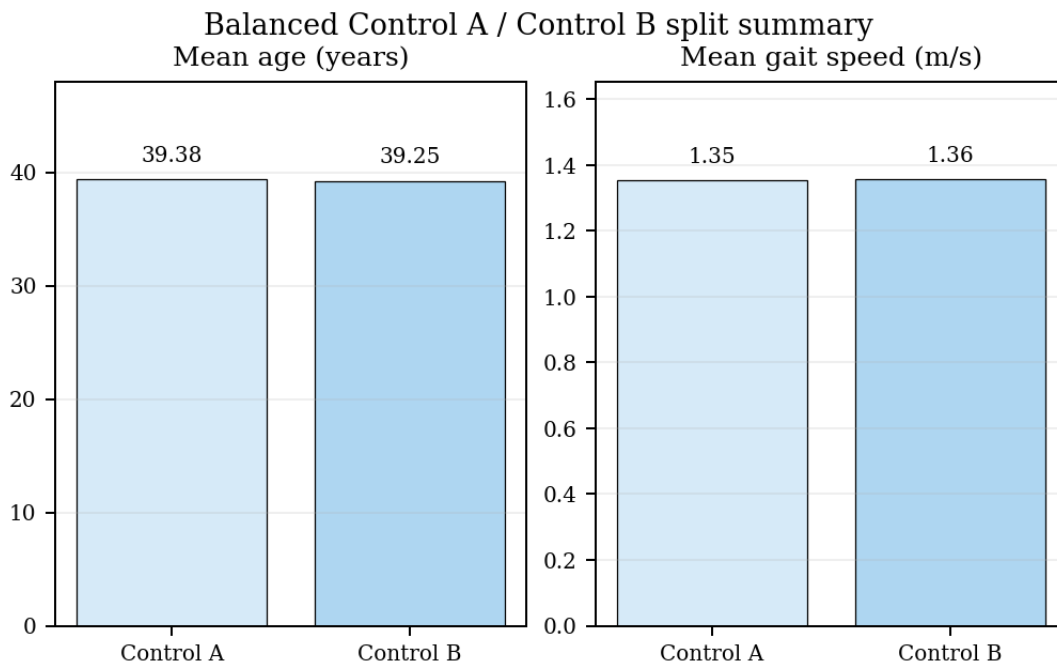
Saved figure: report/figures/v4/pdf/step1/step1_feature_family_inventory.pdf

Saved figure: report/figures/v4/png/step1/step1_feature_family_inventory.png



Saved figure: report/figures/v4/pdf/step1/step1_control_split_summary.pdf

Saved figure: report/figures/v4/png/step1/step1_control_split_summary.png



1.3 Leakage-prevention summary

The main methodological point of Step 1 is not aesthetic organization but boundary control. The feature matrix, subject-level matrix, control partition, and related manifests are frozen before downstream model selection and then reused consistently by Steps 2 through 8. That supports the leakage-control claim for the v4 line, while Step 8 reminds us of the remaining caveat: a frozen split can still produce sensitivity, which is why the completed control-partition analysis must be reported as a sensitivity study rather than as proof of invariance.

The downstream execution manifest and later selection outputs inherit the subject-primary ranking rule `subject_probability_loss_then_lexicographic`, so the preprocessing layer should be understood as the frozen input surface for that downstream decision policy.